

TOKENS

MAGISTRAL · UNIDAD III · WEB 3.0

DE LA FICHA AL DIPLOMA, Y SU ECOSISTEMA

Magistral que une tres temas: token fungible, token único e infraestructura que los vuelve una dApp. La práctica, el viernes 6.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Hoy es clase magistral: la teoría de tres sesiones (S10, S11 y S13) en un solo hilo. No hay laboratorio hoy; la práctica integrada es el viernes 6 de noviembre.

A	**TOKENS FUNGIBLES · ERC-20** Qué es un estándar, la interfaz, approve/transferFrom, decimales, aprobaciones peligrosas, extensiones, Permit y tokenómica.	~65 min
B	**NO FUNGIBLES · NFT, IPFS Y EL CASO USB** ERC-721/1155, tokenURI, dónde vive el archivo, IPFS, CID, pinning, regalías, casos serios y el diploma de la USB (SBT, habeas data).	~70 min
C	**INFRAESTRUCTURA DEL ECOSISTEMA** Leer la historia, The Graph, ENS, firmas EIP-712, SIWE, abstracción de cuentas y dónde vive el frontend.	~55 min

EL PROPÓSITO DE LA SESIÓN

AL SALIR DE AQUÍ

Entender los estándares de token (ERC-20 y ERC-721/1155), dónde vive su metadata y la capa que conecta con una dApp.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

01 ¿Por qué la interoperabilidad de un token depende de un estándar y no del mejor código?

02 ¿Dónde está «el NFT» y dónde está «el archivo», y por qué una URL normal rompe la promesa?

03 ¿Qué firma una billetera en EIP-712, y cómo se lee la historia de un contrato sin recorrer la cadena?

04 ¿En qué se parece «tener un token» a «tener un archivo», y qué los conecta con una pantalla?

RA3 · Programar contratos aplicando estándares de la industria · RA6 · Integrar una interfaz Web 3.0 con contratos on-chain.

SIGLAS Y TÉRMINOS DE HOY

ERC / EIP — *Ethereum Request for Comments / Improvement Proposal*

Un EIP propone; cuando define una interfaz de aplicación, se le llama ERC. ERC-20 es el número 20.

FUNGIBLE — *intercambiable*

Cada unidad es idéntica a otra: un punto vale igual que cualquier otro. Como el dinero. Un NFT es lo contrario.

ALLOWANCE — *autorización · approve*

Cuántos tokens de A puede mover un tercero C. La base de approve + transferFrom.

DECIMALES — *decimals*

Cuántas cifras minúsculas tiene el token. 18 es lo habitual. No hay coma flotante real.

TOKENURI — *enlace a la metadata*

Función que devuelve DÓNDE está el JSON que describe un NFT. Es un texto, no el archivo.

CID · IPFS — *Content Identifier*

El identificador de IPFS: se calcula del contenido. Cambia un byte y cambia el CID.

SBT — *Soulbound Token · ERC-5192*

Token atado a una cuenta: no se puede transferir. Pensado para credenciales.

EIP-712 · SIWE — *firma tipada · login*

Firmar datos con tipos y un dominio; SIWE (ERC-4361) inicia sesión firmando texto, sin contraseña.

A

APROXIMADAMENTE 65 MINUTOS

TOKENS FUNGIBLES · ERC-20

ERC-20 son seis funciones y dos eventos. Que sean SIEMPRE los mismos es justo lo que hace que un token sirva fuera de su propio código.

FUNGIBLE: CADA UNIDAD VALE POR IGUAL

Fungibilidad es que una unidad se puede intercambiar por otra idéntica sin perder valor. Es lo que separa «dinero» de «cosa única», y decide cuál de los dos estándares de hoy le sirve a cada caso.

EJEMPLO	QUÉ LO HACE ASÍ	LA PREGUNTA QUE TIENE SENTIDO
Un billete de 50 000 · un gramo de oro · un punto FUSB	Toda unidad es idéntica e intercambiable.	«¿cuánto tienes?»
La entrada fila 7 asiento 12 · un diploma a tu nombre · la obra original	Cada unidad es distinguible y única.	«¿de quién es el número 7?»

[-] LOS DOS ESTÁNDARES DE HOY

ERC-20 para lo fungible (bloque A) y ERC-721/1155 para lo único (bloque B). El mismo mecanismo —una tabla dentro de un contrato—, dos preguntas distintas: «cuántos» frente a «cuál».

UN ACUERDO, NO UNA LEY

Un EIP (Ethereum Improvement Proposal) es una propuesta pública. Cuando define cómo deben hablar las aplicaciones entre sí, se le llama ERC. Nadie obliga a seguirlos; se siguen porque convienen.

	QUÉ ES	EJEMPLO
EIP	La propuesta, con su número y su discusión abierta.	EIP-20, EIP-721, EIP-2612
ERC	Un EIP que fija una interfaz de aplicación (el «RC» de Request for Comments).	ERC-20 = el EIP-20
El número	Orden de llegada de la propuesta. No hay jerarquía en el número.	20 fue de los primeros

[-] POR QUÉ ESTO IMPORTA HOY

ERC-20 (EIP-20) lo propusieron Fabian Vogelsteller y Vitalik Buterin en noviembre de 2015. Está en estado «Final»: es un contrato social estable del que dependen billeteras, exchanges y todo DeFi. Programar contra un estándar Final es programar contra algo que no se va a mover bajo tus pies.

DE UNA IDEA A «FINAL»: EL PROCESO EIP

Un EIP no se aprueba por votación ni lo decreta una empresa. Avanza por estados públicos, definidos en el EIP-1, con discusión abierta y editores que revisan la forma, no el gusto.



Salidas laterales: «Stagnant» (6 meses sin actividad), «Withdrawn» (el autor la retira) y «Living» (documentos que nunca se congelan, como el propio EIP-1).

EJEMPLO REAL Y ERROR TÍPICO

EIP-20 recorrió ese camino y hoy está en «Final». El EIP-2612 (Permit) también. Error típico: citar un EIP en «Draft» como si fuera estándar. Antes de construir sobre un EIP, miren su estado en eips.ethereum.org: si es Draft o Review, la interfaz todavía puede cambiar bajo sus pies.

SEIS FUNCIONES Y DOS EVENTOS

```
interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address cuenta) external view returns (uint256);
    function transfer(address a, uint256 monto) external returns (bool);

    function allowance(address dueno, address gastador) external view returns (uint256);
    function approve(address gastador, uint256 monto) external returns (bool);
    function transferFrom(address de, address a, uint256 monto) external returns (bool);

    event Transfer(address indexed de, address indexed a, uint256 monto);
    event Approval(address indexed dueno, address indexed gastador, uint256 monto);
}
```

Eso es todo. Un token ERC-20 es cualquier contrato que implemente exactamente estas firmas. OpenZeppelin ya lo hace, bien y auditado: nosotros no reescribimos nada de esto.

QUÉ HACE CADA FUNCIÓN, Y QUIÉN LA LLAMA

FUNCIÓN	QUÉ HACE	QUIÉN LA LLAMA	EL ERROR TÍPICO
`totalSupply()`	Cuántas unidades existen en total.	Cualquiera (lectura).	Leerla sin dividir por 10^decimals.
`balanceOf(c)`	Cuánto tiene la cuenta c.	Billeteras, exploradores.	Creer que el saldo está en la billetera.
`transfer(a, v)`	Mueve v de MÍ (msg.sender) hacia a.	El dueño de los tokens.	Mandar a la dirección del propio contrato: se pierden.
`approve(g, v)`	Autoriza a g a mover hasta v de lo MÍO.	El dueño, antes de usar un contrato.	Aprobar de más «por comodidad».
`allowance(d, g)`	Cuánto le queda a g por mover de d.	Contratos y apps.	Olvidar que baja al gastarse.
`transferFrom(d, a, v)`	g mueve v de d hacia a, dentro de lo aprobado.	El gastador (un contrato).	Llamarla sin approve previo: revierte.

Y dos eventos: Transfer (cada movimiento, incluidos acuñar y quemar, con la dirección cero) y Approval (cada autorización). Los exploradores reconstruyen los saldos leyendo esos eventos.

UN TOKEN NO ES UNA CUENTA CON MONEDAS DENTRO

El error mental más común: creer que cada usuario «tiene» tokens en su billetera. No es así.

```
mapping(address => uint256) private _balances;   // TODO el token es esta tabla
```

DÓNDE VIVE EL TOKEN

En UN contrato: la FichaUSB. Ese contrato guarda una tabla de saldos. Su billetera no contiene tokens: contiene la clave que autoriza a mover una fila de esa tabla.

QUÉ SIGNIFICA «TRANSFERIR»

Restar de una fila y sumar a otra, dentro del mismo contrato. No se mueve nada de un lado a otro: se reescribe una tabla. Por eso una transferencia emite el evento Transfer.

[-] LA CONSECUENCIA PRÁCTICA

Para que una billetera «vea» un token, hay que darle la dirección del contrato: no está en la billetera, está en el contrato.

LA TABLA DE SALDOS, ANTES Y DESPUÉS

Todo el token son dos tablas: balances y allowances. Sigamos a Ana, a Beto y al contrato Canje (precio 25) a lo largo de cuatro operaciones. Cifras en FUSB; dentro del contrato, por 10^{18}.

OPERACIÓN	ANA	BETO	TESORERÍA	ALLOWANCE ANA→CANJE	TOTALSUPPLY
0 · acunar(ana, 100)	100	0	0	0	100
1 · ana.transfer(beto, 30)	70	30	0	0	100
2 · ana.approve(canje, 25)	70	30	0	25	100
3 · canje.canjear() (transferFrom)	45	30	25	0	100
4 · beto.burn(10)	45	20	25	0	90

[-] QUÉ HAY QUE NOTAR

approve NO mueve tokens (fila 2): solo escribe en la tabla de allowances. Y solo acuñar y quemar cambian totalSupply.

POR QUÉ 1 TOKEN SON 1 000 000 000 000 000 000

La EVM no tiene coma flotante (Sesión 5). Un token con 18 decimales guarda internamente enteros enormes, y la coma es pura presentación.

LO QUE VE LA PERSONA	LO QUE GUARDA EL CONTRATO (UNIDADES MÍNIMAS)
1 FUSB	1 000 000 000 000 000 000 (1 × 10^{18})
0,5 FUSB	500 000 000 000 000 000
0,000000000000000001 FUSB	1 (la unidad mínima, indivisible)

⚠ EL ERROR DE PRINCIPIANTE

Escribir acunar(ana, 100) pensando en 100 tokens, cuando la función espera unidades mínimas, acuña una fracción invisible (100 de 10^{18}). Por eso el token recibe la cantidad EN TOKENS y multiplica por 10**decimals() adentro: la interfaz habla en tokens, el contrato en unidades mínimas. En ethers.js se usa parseUnits("1", 18) para escribir y formatUnits(v, 18) para leer.

PARSEUNITS Y FORMATUNITS, CON NÚMEROS REALES

En JavaScript nunca se multiplica a mano por 10^{18}: se usan dos funciones de ethers. parseUnits va de lo humano al contrato; formatUnits, del contrato a lo humano. Salidas reales, ejecutadas con ethers 6:

ETHERS 6 · SALIDA REAL

```
parseUnits("25", 18)         ->  25000000000000000000n   // el precio del Canje
parseUnits("0.1", 18)        ->  100000000000000000n
parseUnits("1000000", 18)    ->  1000000000000000000000000n   // el tope
parseUnits("1.5", 6)         ->  1500000n                 // un token de 6 decimales

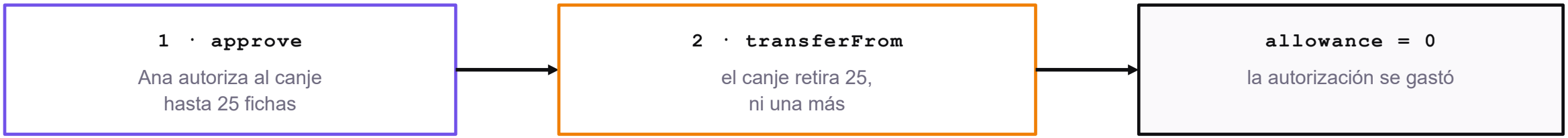
formatUnits(1234567890000000000n, 18)  ->  "123.456789"
formatUnits(100n, 18)        ->  "0.000000000000000001"     // el error de A.8
```

⚠ TRES ERRORES TÍPICOS

(1) Usar Number en vez de BigInt: 25e18 no cabe exacto en un Number de JavaScript. (2) Suponer 18 decimales en un token de 6. (3) Pasar un string con coma («0,5»): parseUnits exige punto decimal.

POR QUÉ TRANSFERIR EN DOS PASOS

transfer mueve MIS tokens. Pero, ¿cómo deja un usuario que un CONTRATO —un exchange, un canje— mueva los suyos, sin entregarle su clave privada? Con una autorización previa y acotada.



LO QUE HACE SEGURO EL PATRÓN

El contrato NUNCA puede mover más de lo aprobado. Verificado en el laboratorio: un segundo canje sin volver a aprobar revierte. La clave privada de Ana no sale nunca de su billetera.

Así funcionan todos los intercambios descentralizados: primero autorizas al contrato del DEX, después el DEX mueve tus tokens al hacer el swap.

QUÉ PASA EXACTAMENTE EN CANJEAR()

```
function canjear() external {
    // el Canje es msg.sender para la ficha: es el GASTADOR
    bool ok = ficha.transferFrom(msg.sender, tesoreria, precio);
    if (!ok) revert TransferenciaFallida();

    premiosDe[msg.sender] += 1;
    emit Canjeado(msg.sender, precio);
}
```

- Ana llama canjear(): para el Canje, msg.sender = Ana.
- El Canje llama a la ficha: para la ficha, msg.sender = el Canje.
- La ficha revisa allowance[Ana][Canje] ≥ 25, la descuenta y mueve 25 de Ana a la tesorería.
- Si no alcanza, revierte TODO: no hay premio sin pago.

⚠ **EL ERROR TÍPICO DE QUIEN LO PROGRAMA**

Escribir ficha.transferFrom(tesoreria, msg.sender, precio) —al revés—, o usar ficha.transfer(tesoreria, precio): eso intentaría mover los tokens DEL CANJE, que no tiene ninguno. El «de» de transferFrom es el usuario; quien gasta la autorización es el contrato que hace la llamada. Tampoco hace falta un require sobre la allowance: transferFrom ya revierte con ERC20InsufficientAllowance.

APROBACIONES ILIMITADAS: COMODIDAD PELIGROSA

Muchas aplicaciones piden aprobar una cantidad gigante «para no molestar otra vez». Es cómodo, y es el origen de robos enormes.

QUÉ PASA

Si apruebas una cantidad ilimitada a un contrato, y ese contrato resulta malicioso o es hackeado más adelante, puede vaciar TODO tu saldo de ese token cuando quiera. La autorización sigue viva hasta que la revoques.

QUÉ HACER

Aprobar solo lo necesario para la operación. Revisar y revocar aprobaciones viejas (hay herramientas para verlas todas). Desconfiar de una app que pide aprobación ilimitada sin explicarlo.

EL CASO REAL · BADGER DAO, DICIEMBRE DE 2021

El front-end de Badger DAO fue comprometido (un script malicioso inyectado vía Cloudflare) y pedía a los usuarios firmar aprobaciones ILIMITADAS. Unas 500 billeteras aprobaron; el atacante usó transferFrom y drenó del orden de 120 millones de dólares. El contrato estaba bien: el robo entró por una aprobación excesiva que la gente firmó sin mirar. Regla: se entrega la aprobación más pequeña que sirva, y se revoca poniendo la allowance en cero.

CAMBIAR UNA APROBACIÓN DE 50 A 20

El propio EIP-20 advierte de un riesgo de carrera. Ana aprobó 50 a Beto (un gastador) y decide bajarlo a 20. Parece inofensivo. Pero su transacción pasa por el mempool, a la vista (Sesión 9):

01	ANA ENVÍA APPROVE(BETO, 20)	Su transacción queda pendiente en el mempool. La allowance todavía vale 50.
02	BETO LA VE Y SE ADELANTA	Envía transferFrom(ana, beto, 50) con más gas: entra ANTES. Se lleva los 50 viejos.
03	SE CONFIRMA LA DE ANA	approve escribe 20 (no resta: SOBRESCRIBE). Beto tiene una allowance nueva de 20.
04	BETO GASTA OTRA VEZ	transferFrom(ana, beto, 20). Total: 70. Ana quería autorizar como máximo 50.

LA IDEA LLANA

approve no dice «cambia de 50 a 20»; dice «ahora son 20», sin saber cuánto se gastó ya.

CÓMO CAMBIAR UNA APROBACIÓN SIN REGALAR

OPCIÓN	CÓMO FUNCIONA	TRADE-OFF
Pasar por 0	approve(g, 0), confirmar, mirar cuánto gastó g, y recién entonces approve(g, M).	Dos transacciones, pero decides M sabiendo lo gastado. Lo recomienda el EIP-20.
increase / decreaseAllowance	Sumar o restar a la allowance en vez de sobrescribirla.	Existían en OpenZeppelin 4. Se QUITARON de ERC20 en OZ 5 (no son del estándar y se usaron en phishing).
SafeERC20 (lado del contrato)	forceApprove y safeIncreaseAllowance, para contratos que aprueban a otros.	Es una librería para quien LLAMA al token, no una función del token.
Aprobar exacto y a tiempo	Aprobar solo lo que se va a gastar, justo antes de usarlo (o con Permit).	La allowance vive poco y vale poco: casi nada que correr.

⚠ ERROR TÍPICO: COPIAR UN TUTORIAL VIEJO

Muchos tutoriales llaman token.increaseAllowance(...). Con OpenZeppelin 5.6.1 (el del curso) eso no compila: la función ya no existe en ERC20.

QUÉ AÑADE CADA PIEZA DE OPENZEPPELIN

EXTENSIÓN	QUÉ AÑADE	CUÁNDO USARLA
ERC20Burnable	burn y burnFrom: destruir tokens propios o aprobados.	Puntos que se canjean o retiran de circulación.
ERC20Capped	Un tope máximo de emisión que nunca se supera.	Cuando prometes escasez: oferta con techo.
ERC20Pausable	Congelar todas las transferencias ante una emergencia.	Si necesitas un «freno» (con su propio riesgo de centralización).
ERC20Permit (EIP-2612)	Aprobar con una FIRMA fuera de la cadena, sin transacción de approve.	Mejor experiencia: una sola transacción hace todo.

COMPONER, NO REESCRIBIR

Se heredan las que se necesitan y se combinan. Cada una está auditada por separado. La habilidad no es escribir un ERC-20 desde cero —eso es un antipatrón— sino elegir y componer las extensiones correctas para tu problema.

BURNABLE, CAPPED Y PAUSABLE: LOS TRADE-OFFS

EXTENSIÓN	LO QUE GANAS	LO QUE CUESTA O ARRIESGA	EJEMPLO EN FICHAUSB
Burnable	Sacar unidades de circulación: canjes, retiros, deflación.	La quema es irreversible; burnFrom exige allowance (otra aprobación que vigilar).	Ana quema 40 → balance y totalSupply bajan 40.
Capped	Escasez creíble: nadie puede emitir por encima del tope, ni el dueño.	Si el tope es muy bajo, el proyecto se queda sin emisión. Cambiarlo exige otro contrato.	Tope 1 000 000: acuñar 1 más revierte con ERC20ExceededCap.
Pausable	Un freno de emergencia: congelar todas las transferencias.	Quien tiene el freno puede congelar el dinero de todos: es poder centralizado.	FichaUSB NO la usa, a propósito: no hay freno que abusar.

⚠ **ERROR TÍPICO CON CAPPED**

Creer que el tope cuenta lo acuñado en la historia. Cuenta el suministro VIVO: si se queman 10 con el tope lleno, se pueden volver a acuñar 10. Lo demuestra la prueba ★ «quemar libera espacio bajo el tope».

EIP-2612 PERMIT: UN PASO MENOS

El patrón clásico exige DOS transacciones: aprobar y luego usar. Permit deja aprobar con una FIRMA fuera de la cadena, que el contrato verifica al usarla. El usuario firma gratis; una sola transacción hace todo.

SIN PERMIT

- Transacción 1: approve (cuesta gas).
- Transacción 2: la operación (cuesta gas).
- Dos confirmaciones en la billetera.

CON PERMIT (EIP-2612)

- Una firma fuera de la cadena (gratis, sin gas).
- Transacción única: la operación incluye la aprobación.
- La firma usa EIP-712, que veremos en el bloque C.

OpenZeppelin lo trae en la extensión ERC20Permit. Lo propuso Martin Lundfall en 2020, es estándar «Final», y hoy lo usan casi todos los tokens grandes. En el proyecto es opcional, pero mejora mucho la experiencia.

EL FLUJO DE UNA APROBACIÓN FIRMADA

01	ANA FIRMA, SIN GAS	Su billetera muestra un mensaje estructurado (EIP-712): dueño, gastador, valor, nonce y fecha límite. Firma fuera de la cadena.
02	LA APP RECIBE LA FIRMA	La firma (v, r, s) viaja a la aplicación. Todavía no ha pasado nada en la cadena.
03	UNA SOLA TRANSACCIÓN	El contrato llama token.permit(ana, gastador, valor, deadline, v, r, s) y, a continuación, transferFrom.
04	EL TOKEN VERIFICA	Recupera quién firmó, compara con ana, revisa el deadline y consume el nonce: la misma firma no se puede reusar.

SI ALGO NO CUADRA

Firma vencida: ERC2612ExpiredSignature. Firmante distinto del dueño: ERC2612InvalidSigner. El DOMAIN_SEPARATOR ata la firma a esa red y a ese contrato.

EL TRADE-OFF

Firmar un permit ES aprobar, aunque no cueste gas. «Es gratis, solo una firma» es justo lo que usa el phishing: se lee el gastador y el valor antes de firmar.

EL MISMO CÓDIGO, DOS INTENCIONES

Los dos tokens siguientes son ERC-20 perfectamente válidos, heredados de OpenZeppelin. Lo que los separa no es la calidad del código: es a quién le sirven las reglas.

DECISIÓN	A · SOSTENIBLE (FICHAUSB)	B · EXTRACTIVO (EJEMPLO ILUSTRATIVO)
Suministro	Tope fijo de 1 000 000, visible en cap().	Sin tope: mint() del dueño sin límite.
Emisión	El organizador acuña por asistencia, a la vista (eventos Transfer).	El dueño acuña cuando quiere, sin regla pública.
Distribución	Se reparte a quien participa; el equipo no se queda un lote.	90 % preasignado al equipo, sin vesting.
Quema	Cualquiera quema lo suyo al canjear un premio.	Sin quema; o el dueño quema saldos AJENOS.
Poderes especiales	Ninguno más allá de acuñar bajo el tope.	Pausa, lista negra y comisión de venta que el dueño cambia a su antojo.

LA PREGUNTA QUE DECIDE

¿Quién puede cambiar las reglas después de que tú compraste o recibiste el token?

CÓMO LEER UN TOKEN ANTES DE CONFIAR EN ÉL

Todo esto se puede verificar en el explorador, con el código verificado delante. Verificar en vez de creer (la tesis del curso) aplicado a tokens:

SEÑAL DE ALERTA	CÓMO SE VE EN EL CÓDIGO O EN LA CADENA
El dueño imprime sin tope	Una función de mint con onlyOwner y sin cap(), o con un cap que el dueño cambia.
Concentración inicial	Los primeros eventos Transfer desde 0x0 van casi todos a pocas direcciones.
Sin vesting	El equipo recibe todo el día uno, sin contrato que libere por partes.
Poderes de congelar o confiscar	Pausable, listas negras o funciones que mueven saldos ajenos en manos de una sola llave.
Código no verificado	Si no se puede leer el código en el explorador, no hay nada que auditar: se descarta.

⚠ ERROR TÍPICO

Juzgar el token por el sitio web o por el precio. Se juzga por el contrato y por la distribución real en la cadena.

SEIS IDEAS SOBRE TOKENS

01 Un token vale por cumplir el estándar (EIP-20), no por tener el mejor código.

02 Todo el token es una tabla de saldos dentro de UN contrato.

03 Se hereda y se COMPONE de OpenZeppelin; casi nada se escribe a mano.

04 No hay decimales reales: la interfaz habla en tokens, el contrato en unidades mínimas.

05 approve + transferFrom deja mover tus tokens sin tu clave, y solo lo aprobado; ilimitado es peligroso.

06 La tokenómica es una decisión ética además de técnica: repartir y limitar, o concentrar.

B

APROXIMADAMENTE 70 MINUTOS

NO FUNGIBLES · NFT, IPFS Y EL CASO USB

Comprar un NFT casi nunca es comprar una imagen. Es comprar una fila en un contrato que apunta a una imagen. Dónde apunta lo cambia todo.

UN BILLETE CUALQUIERA, O ESTA ENTRADA EXACTA

FUNGIBLE · ERC-20 (BLOQUE A)

Cada unidad es idéntica e intercambiable. Mi FUSB vale igual que tu FUSB. Solo importa CUÁNTOS tienes. Como el dinero: da igual qué billete de 50 000 te entreguen.

NO FUNGIBLE · ERC-721

Cada token es único, con identidad propia. Importa CUÁL tienes. Una entrada con asiento asignado, un diploma, un título de propiedad: no se cambian uno por otro.

```
ERC-20:  balanceOf(ana) = 30 FUSB      ·      ERC-721:  ownerOf(7) = ana
```

⚠ **EL ERROR DE CONCEPTO TÍPICO**

«No fungible» no significa «valioso» ni «artístico». Significa solo que cada unidad tiene identidad. Un NFT puede no valer nada y un token fungible puede valer mucho.

UN NFT ES UNA FILA EN UNA TABLA

Por dentro, un ERC-721 es poco más que dos diccionarios. Este es el estado real de nuestro DiplomaUSB después de emitir dos diplomas en la práctica del viernes 6:

TOKENID	OWNEROF(TOKENID)	TOKENURI(TOKENID)
1	0x7099…79C8 (Ana)	ipfs://bafkrei…/1.json
2	0x3C44…93BC (Beto)	ipfs://bafkrei…/2.json
3	— revierte: ERC721NonexistentToken(3)	— revierte

QUÉ ES «TENER» EL NFT

Que tu dirección aparezca en la columna ownerOf. Nada más. Transferirlo es cambiar esa celda, con tu firma.

QUÉ NO ESTÁ EN LA TABLA

La imagen, el nombre, la descripción. Solo hay un texto que dice dónde buscarlos: el tokenURI.

LAS FUNCIONES QUE TODO NFT TIENE

```
// ERC-721 (EIP-721, 2018) · lo mínimo que exige el estándar
function balanceOf(address dueno) view returns (uint256);
function ownerOf(uint256 tokenId) view returns (address);
function transferFrom(address de, address a, uint256 tokenId);
function safeTransferFrom(address de, address a, uint256 tokenId);
function approve(address operador, uint256 tokenId);
function setApprovalForAll(address operador, bool aprobado);
event Transfer(address indexed de, address indexed a, uint256 indexed tokenId);
// extensión de metadata (la que usan los exploradores)
function tokenURI(uint256 tokenId) view returns (string);
```

- balanceOf dice CUÁNTOS NFT tienes; ownerOf dice de quién es UNO.
- safeTransferFrom comprueba que un contrato receptor sepa recibir NFT.
- Emitir (mint) es un Transfer desde la dirección cero.

SETAPPROVALFORALL: LA FIRMA MÁS PELIGROSA DEL ECOSISTEMA

Autoriza a un operador a mover TODOS tus NFT de ese contrato. Es lo que piden los sitios de phishing. No se firma lo que no se entiende.

ERC-1155: MUCHOS TIPOS EN UN CONTRATO

En ERC-1155 el saldo depende de dos cosas: la cuenta Y el id. Cada id puede tener una sola copia (no fungible) o miles (fungible). Y se pueden mover varios ids en una sola transacción.

```
balanceOf(ana, 1)    // id 1: "Espada legendaria" -> 1 (única)
balanceOf(ana, 2)    // id 2: "Poción"            -> 250 (fungible)
safeBatchTransferFrom(ana, beto, [1, 2], [1, 10], "")
// una transacción: la espada y 10 pociones
uri(2) -> "ipfs://bafy.../{id}.json"   // la app reemplaza {id}
```

⚠ **EL ERROR DE CONCEPTO TÍPICO**

«1155 es un 721 mejorado.» No: es otro modelo. Al permitir copias, un id de 1155 no identifica a UNA pieza; si necesitan una pieza con historia propia, 1155 los obliga a usar un id por pieza, y ahí pierde su ventaja.

ERC-721 FRENTE A ERC-1155

CRITERIO	ERC-721	ERC-1155
Unidad	Un token único por id.	Un id con N copias (N puede ser 1).
Saldo	ownerOf(id) → una dirección.	balanceOf(cuenta, id) → una cantidad.
Transferir varios	Uno por llamada.	Por lotes: muchos ids en una transacción.
Caso típico	Diploma, obra única, título.	Ítems de juego, entradas por categoría.
Soporte en exploradores	Universal.	Amplio, algo menor.

[-] LA REGLA PRÁCTICA

¿Cada ítem tiene historia propia (quién lo tuvo, cuándo)? ERC-721. ¿Hay muchas copias idénticas de pocos tipos, o se mueven inventarios en lote? ERC-1155. Si dudan, empiecen por 721: es el más simple de explicar y de auditar.

LA FICHA QUE LEEN EXPLORADORES Y BILLETERAS

DIPLOMA.JSON · SUBIDO A IPFS

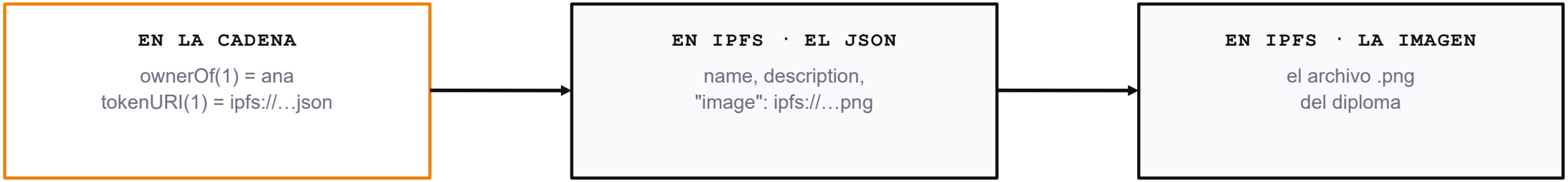
```
{
  "name": "Diploma de prueba · Laboratorio 11",
  "description": "Credencial de PRUEBA. Sin datos personales.",
  "image": "ipfs://bafybei.../diploma.png",
  "attributes": [
    { "trait_type": "Programa", "value": "Ingeniería de Sistemas" },
    { "trait_type": "Cohorte",  "value": "2026-2" }
  ]
}
```

- name, description e image vienen del esquema de EIP-721.
- attributes no es del estándar: es una convención que popularizó OpenSea y hoy leen casi todos.
- image es OTRO enlace IPFS: la ficha y la imagen son dos archivos.

REGLA DEL CURSO · HABEAS DATA

Ni nombre, ni cédula, ni correo en el JSON: en IPFS es público y no se puede borrar. Ley 1581 de 2012. Un diploma real publicaría, a lo sumo, el hash del documento (como en la Sesión 6).

«EL NFT» NO ES «EL ARCHIVO»



Tres saltos, dos redes. El contrato guarda al dueño y UN texto. Ese texto lleva al JSON; el JSON lleva a la imagen. Un explorador que «muestra el NFT» está haciendo estos tres saltos por ustedes.

⚠ LO QUE DE VERDAD SE COMPRA

Comprar un NFT es quedar en la celda ownerOf. Si la imagen desaparece, siguen siendo dueños de un token que apunta a la nada. Y tener el NFT no da derechos de autor sobre la imagen: eso lo decide una licencia, no el contrato. En la práctica del viernes 6 recorrerán los tres saltos a mano con el script leer-diploma.js.

MEDIDO: GUARDAR BYTES CUESTA UNA FORTUNA

Lo medimos en Hardhat con un contrato que solo hace datos = d (almacenamiento permanente en la cadena):

QUÉ SE GUARDA	GAS MEDIDO	COMENTARIO
1 KB de bytes	770 543	Ya es 15 veces una transferencia de FUSB (51 692).
20 KB de bytes	14 570 319	Unos 730 000 de gas por KB.
Imagen de 100 KB	≈ 73 millones (extrapolado)	No cabe en un bloque de la red local de Hardhat (60 millones).
El tokenURI (67 caracteres)	173 850 · emitir completo	Incluye crear el token, el dueño y el enlace.

[-] LA CONCLUSIÓN DE INGENIERÍA

En la cadena va lo que necesita consenso: quién es el dueño y un compromiso con el contenido (el CID). El contenido pesado va afuera. Hay NFT «100 % on-chain» (SVG pequeños generados por código), pero son la excepción.

UNA URL NORMAL ROMPE LA PROMESA

`TOKENURI = HTTPS://…`

https://miuni.edu/diplomas/1.json

Quien controla ese servidor puede cambiar el archivo mañana sin tocar la cadena. Si el dominio vence o el servidor cae, el NFT apunta a la nada. La cadena es inmutable; el servidor, no.

`TOKENURI = IPFS://CID`

ipfs://bafkrei…

El enlace ES el hash del contenido. Si alguien cambia un byte, el CID ya no coincide: cualquiera lo detecta. Nadie puede sustituir el contenido a escondidas, ni siquiera quien lo subió.

IPFS no hace eterno un archivo. Hace imposible cambiarlo sin que se note. Son dos garantías distintas.

PEDIR POR CONTENIDO, NO POR UBICACIÓN

LA WEB · POR UBICACIÓN

«Tráeme lo que haya en ese servidor, en esa ruta.» Lo que llega depende de quién controla el servidor hoy.

IPFS · POR CONTENIDO

«Tráeme el archivo cuyo hash es este.» Cualquier nodo que lo tenga lo sirve, y el que lo recibe recalcula el hash para comprobarlo.

01	PEDIR	El navegador o la puerta (gateway) pregunta a la red: ¿quién tiene el CID bafkrei…?
02	ENCONTRAR	Una tabla distribuida (DHT) responde qué nodos anunciaron tener ese contenido.
03	DESCARGAR	Se baja de cualquiera de ellos: no importa de cuál, porque…
04	VERIFICAR	…se recalcula el hash. Si no coincide con el CID, se descarta. Confianza cero en el nodo.

EL CID ES EL HASH, CON UNA ETIQUETA DELANTE

SALIDA REAL DEL SCRIPT CID-RAW.JS

```
node scripts/s11/cid-raw.js --texto "hello world"
  sha256  : b94d27b9934d3e08a52e52d7da7dabfac484efe3...
  CID raw : bafkreifzjut3te2nhyekklss27nh3k72ysco7y32koao5eei66wof36n5e

--texto "Diploma USB - prueba 1" -> bafkreiez36qzdmqn6bh6gefu6wpky4dezu...
--texto "Diploma USB - prueba 2" -> bafkreicmjjso6teuztsxwcuwv4stofvsen...
```

`CID v1 raw = "b" + base32( 01 · 55 · 12 · 20 · sha256(bytes) )` versión · raw · sha2-256 · 32 bytes

COMPRUÉBENLO EN VIVO

https://ipfs.io/ipfs/bafkreifz…n5e devuelve «hello world». Nadie lo subió para esta clase: alguien en el mundo fijó esos mismos bytes y el CID coincide.

EL ERROR DE CONCEPTO TÍPICO

«Mismo archivo, mismo CID siempre.» No: Pinata puede dar un CID «bafybei…» para esos bytes porque los envuelve en otra estructura. Ambos son hash; cambia la receta.

PINNING: ALGUIEN TIENE QUE GUARDARLO

Un nodo IPFS guarda lo que visita solo como caché y lo borra cuando necesita espacio. Un archivo sobrevive si algún nodo lo FIJA. Los servicios de pinning son empresas que fijan por ustedes.

SERVICIO	PLAN GRATUITO ANUNCIADO (SEP. 2026)	PARA LA PRÁCTICA
Pinata	1 GB, 500 archivos, una puerta propia.	Opción principal. Subir como PÚBLICO.
Filebase	5 GB, compartidos con su almacenamiento S3.	Alternativa si Pinata falla.
IPFS Desktop	Su propio nodo, gratis.	Plan B: el archivo existe solo mientras su PC esté encendido.

⚠ DOS TRAMPAS QUE VIMOS AL PREPARAR LA CLASE

1) Pinata permite subir como «privado»: esos archivos NO se anuncian a la red y ninguna puerta pública los encuentra. 2) Las puertas públicas (ipfs.io, dweb.link) limitan consultas y a veces responden 429: prueben con la puerta de su propio servicio.

SERVIDOR, IPFS O ARWEAVE

	SERVIDOR HTTPS	IPFS + PINNING	ARWEAVE
Integridad	Ninguna: el dueño cambia el archivo.	Sí: el CID es el hash.	Sí: identificador ligado al contenido.
Disponibilidad	Mientras paguen el servidor.	Mientras alguien fije (y pague).	Pago único, diseñado para ser permanente.
Costo	Mensual.	Mensual (gratis en poco volumen).	Una vez, al subir.
Riesgo principal	Cambio o caída silenciosa.	Que nadie siga fijando.	Depende de que su economía se sostenga.

VERIFICACIÓN · ALMACENAMIENTO

La universidad sube el JSON a IPFS y deja de pagar Pinata en 2030. ¿Qué pasa con el diploma en la cadena? ¿Y qué pasa si en vez de eso alguien intenta cambiar el JSON? Respondan las dos por separado.

EIP-2981: SE DECLARA, NO SE GARANTIZA

```
function royaltyInfo(uint256 tokenId, uint256 precioVenta)
    external view returns (address receptor, uint256 regalia);
// ej.: 5 % -> royaltyInfo(1, 1 ether) = (creador, 0.05 ether)
```

El propio estándar lo dice: el pago es voluntario, porque transferFrom no sabe si hubo una venta. El contrato informa; el marketplace decide si paga.

FECHA	QUÉ PASÓ
Nov. 2022	OpenSea lanza su «Operator Filter» para bloquear mercados que no pagan regalías.
Ago. 2023	OpenSea lo apaga: regalías opcionales para colecciones nuevas desde el 31 de agosto.
2024	OpenSea soporta ERC-721C: el contrato restringe quién puede transferir para forzar el pago.

Forzar regalías exige restringir transferencias. Es el mismo dilema del proyecto de entradas: control del emisor frente a libertad del dueño.

CASOS DONDE LA UNICIDAD SÍ IMPORTA

CASO	QUÉ APORTA EL NFT	EL MATIZ HONESTO
Credenciales académicas	Cualquiera verifica quién emitió qué, sin llamar a la universidad.	¿Debería poder transferirse? El caso USB.
Trazabilidad (café, cacao)	Cada lote con su historia pública e inalterable.	El dato de entrada sigue dependiendo de un humano honesto.
Entradas a eventos	Cada entrada es única; la reventa se acota por contrato.	Es el proyecto del docente.
Licencias y membresías	Acceso verificable y, si se quiere, transferible.	A menudo ERC-1155 encaja mejor.
Activos reales (RWA)	Un bien, un token, un dueño visible.	Sin reconocimiento legal, el token no es el título.

En todos sigue viva la pregunta de la Sesión 1: ¿hace falta una cadena, o basta una base de datos firmada por quien ya es la autoridad?

LO QUE EL MERCADO NFT HIZO MAL

CRÍTICA	EN QUÉ CONSISTE
Enlaces rotos	Colecciones con tokenURI en servidores propios que cerraron: el token queda apuntando a nada.
Tokenizar no da derechos	Cualquiera puede emitir un NFT de una obra ajena. El contrato no verifica autoría.
Especulación y lavado de volumen	Ventas de una cuenta a sí misma para inflar precios (wash trading), documentadas en informes de análisis de cadena.
Phishing de aprobaciones	Sitios falsos que piden setApprovalForAll y vacían la billetera.
Consumo energético	Crítica válida en 2021; en Ethereum bajó drásticamente con la prueba de participación (Sesión 4).

Nada de esto invalida la técnica. Invalida usarla sin preguntarse qué problema resuelve. Esa es la tesis del curso.

TRES FORMAS DE MODELAR EL MISMO DIPLOMA

MODELO	A FAVOR	EN CONTRA
ERC-721 transferible	Estándar, simple; cualquier explorador lo muestra.	Un título se podría vender o regalar: el dueño del token no es el graduado.
ERC-1155 (un id por programa)	Barato para cohortes grandes; un id = «Sistemas 2026».	Las copias son idénticas: no distingue a una persona de otra del mismo programa.
SBT · ERC-5192	No se transfiere: el token queda atado a la cuenta del graduado.	Pierde la billetera, pierde el diploma; la universidad conserva poder de revocar.

ERC-5192 EN UNA LÍNEA

Estándar Final: añade locked(tokenId) y los eventos Locked/Unlocked a un ERC-721. Si locked devuelve true, TODA función que mueva el token entre cuentas debe revertir. Interfaz ERC-165: 0xb45a3c0e.

UN IF EN _UPDATE BLOQUEA LA TRANSFERENCIA

CONTRACTS/S11/DIPLOMASBT.SOL

```
function _update(address to, uint256 tokenId, address auth)
    internal override returns (address)
{
    address from = _ownerOf(tokenId);
    if (from != address(0) && to != address(0))       // ni mint ni quema
        revert DiplomaIntransferible(tokenId);          // = transferencia
    address anterior = super._update(to, tokenId, auth);
    if (from == address(0)) emit Locked(tokenId);      // ERC-5192
    return anterior;
}
```

- En OpenZeppelin v5 todo movimiento pasa por _update.
- Emitir (de 0) y revocar (a 0) siguen permitidos.
- Reto opcional de la práctica: andamiaje/s11/DiplomaSBT.sol.

LO QUE MUESTRA LA PRUEBA «BILLETERA PERDIDA»

La universidad revoca el token 1 y emite el 2 a la cuenta nueva. El 1 queda quemado para siempre. ¿Es eso un diploma «inmutable»? ¿Quién tiene el poder real?

LEY 1581 DE 2012: NADA PERSONAL EN CADENA

DÓNDE	QUÉ SÍ	QUÉ NO, NUNCA
En la cadena	Dirección, tokenId, tokenURI, fecha de emisión.	Nombre, cédula, correo, fotografía.
En IPFS (público)	Programa, cohorte, imagen genérica del diploma.	Cualquier dato que identifique a la persona.
En la universidad	El PDF firmado y la relación persona ↔ dirección.	— (es donde la ley sí permite custodiarlos)

EL CHOQUE

La ley reconoce al titular el derecho a pedir la supresión de sus datos. Una cadena pública y un CID fijado por terceros no se pueden borrar.

LA SALIDA DE DISEÑO

Lo personal nunca sale de la universidad. En cadena, a lo sumo, el hash del PDF (Sesión 6): prueba integridad sin revelar nada.

SEIS IDEAS SOBRE NFTS

01 No fungible = con identidad: importa CUÁL, no cuántos. No significa «valioso».

02 ERC-721 para piezas con historia propia; ERC-1155 para muchas copias de pocos tipos.

03 «El NFT» es la celda ownerOf; el JSON y la imagen viven afuera, casi siempre en IPFS.

04 IPFS garantiza integridad (el CID es el hash), no disponibilidad: eso es el pinning.

05 Las regalías (EIP-2981) se declaran; forzarlas exige restringir transferencias.

06 Firmar setApprovalForAll es entregar todos tus NFT de ese contrato. Leer antes de firmar.

C

APROXIMADAMENTE 55 MINUTOS

INFRAESTRUCTURA DEL ECOSISTEMA

Un contrato y una pantalla bastan para una demo. Para que la use gente real faltan piezas: leer la historia, nombres legibles, firmas y cuentas amables.

LA PLOMERÍA QUE UNA DAPP NECESITA

Con los bloques A y B ya saben escribir el contrato: el token fungible y el único. Pero un contrato solo no es una aplicación. Falta la plomería que lo vuelve usable para una persona de verdad.

LEER
indexar lo que pasó

FIRMAR
identidad sin contraseña

CUENTA
abstracción de cuentas

ALOJAR
dónde vive el frontend

LO QUE TIENEN EN COMÚN

Ninguna de estas piezas cambia el contrato; todas cambian la experiencia de quien lo usa. Este bloque recorre las cuatro, en ese orden, y cierra con qué necesita un repositorio para que otro lo corra.

EL ESTADO SOLO GUARDA EL PRESENTE

Leer una variable como balanceOf es una llamada instantánea y gratis. Pero el producto pide otra cosa: «muéstrame todas las asistencias de Ana» o «las diez últimas transferencias». Y eso el contrato no lo sabe.

LO QUE GUARDA EL ESTADO

- El VALOR ACTUAL de cada variable: saldo de hoy, dueño de hoy.
- Cada escritura pisa la anterior: el valor viejo ya no está en el storage.
- Leerlo es barato: una llamada por dato.

DÓNDE ESTÁ LA HISTORIA

- En los LOGS de los recibos: cada emit deja un registro con el bloque y la transacción.
- Los logs no son accesibles desde el contrato: solo desde fuera.
- Filtrarlos exige pedir rangos de bloques al nodo.

⚠ ERROR TÍPICO

«Guardo un arreglo con todo el historial en el contrato y lo leo desde la interfaz». Funciona con diez registros y se vuelve caro e ilegible con diez mil. La historia va en eventos; el estado guarda solo lo que el contrato necesita para decidir.

RECORRER UN AÑO DE BLOQUES, A MANO

Supongamos que la interfaz quiere el historial completo de un contrato que lleva un año desplegado. En Ethereum sale un bloque cada 12 segundos.

2 628 000

bloques en un año: 365 × 24 × 3 600 ÷ 12. Todos hay que revisarlos buscando los logs del contrato.

263

consultas eth_getLogs si el proveedor deja pedir 10 000 bloques por vez. En cada visita de cada usuario.

0

capacidad de ordenar, agregar o paginar en el nodo: eso lo tiene que hacer el navegador con todo en memoria.

Cada proveedor de RPC limita el rango de bloques o el número de resultados por consulta, y cuenta las llamadas contra su plan gratuito. Lo que en local es instantáneo, en Sepolia o en la red principal se vuelve lento, caro o simplemente rechazado.

La salida es indexar: leer los eventos UNA vez, guardarlos en una base de datos, y que la interfaz consulte esa base.

LEER EVENTOS CON QUERYFILTER

DEL CONTRATO A LA CONSULTA

```
// el evento de la práctica del viernes 6: tres campos indexed = tres «topics» filtrables
event AsistenciaRegistrada(address indexed estudiante, uint256 indexed sesion,
                           address indexed enviadaPor, uint256 nonce);

// ethers v6, desde la interfaz o un script
const filtro  = contrato.filters.AsistenciaRegistrada(ana);    // solo las de Ana
const eventos = await contrato.queryFilter(filtro, bloqueDespliegue, "latest");
for (const e of eventos) console.log(e.blockNumber, e.args.sesion, e.args.nonce);
```

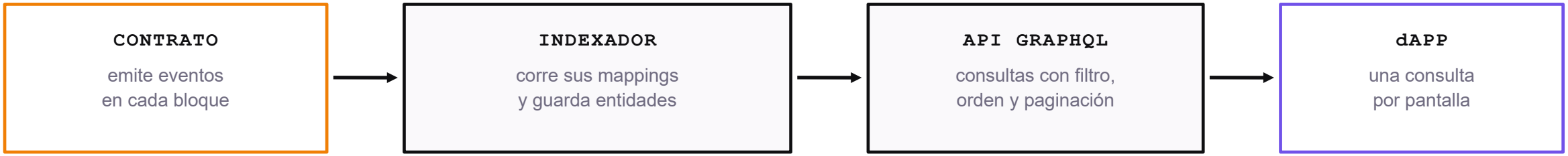
CUÁNDO BASTA

- Pocos eventos y un solo contrato: un proyecto de curso.
- Empezar la búsqueda en el bloque del despliegue, no en 0.
- Filtrar por campos indexed: el nodo hace el trabajo.

CUÁNDO YA NO

- Miles de eventos o varios contratos a la vez.
- Hace falta ordenar, sumar o paginar.
- Cada usuario repetiría las mismas consultas al RPC.

THE GRAPH: UN SUBGRAFO EN TRES ARCHIVOS



ARCHIVO	QUÉ DICE
subgraph.yaml	Qué contrato, en qué red, desde qué bloque y qué eventos escuchar (el manifiesto).
schema.graphql	Qué entidades se guardan: por ejemplo Asistencia { estudiante, sesion, bloque }.
mapping.ts	Qué hacer con cada evento: crear o actualizar entidades. Se escribe en AssemblyScript.

[-] ESTADO ACTUAL, VERIFICADO

Para pocos eventos basta leerlos con queryFilter desde la interfaz. Cuando hay volumen, filtros, orden o varias entidades, se indexa. El «hosted service» gratuito de The Graph se apagó el 12 de junio de 2024; hoy se publica en Subgraph Studio (plan gratuito: 100 000 consultas al mes).

¿CUÁL USAR EN SU PROYECTO?

OPCIÓN	CUÁNDO CONVIENE	QUÉ CUESTA	QUÉ CENTRALIZA
`queryFilter en la interfaz`	Pocos eventos, un contrato, consultas simples.	Nada extra; llamadas al RPC.	Nada nuevo: el mismo RPC de siempre.
`Subgrafo (The Graph)`	Muchos eventos, filtros, orden, varias entidades.	Aprender GraphQL y AssemblyScript; consultas sobre el plan gratuito se pagan.	Dependen de la red de indexadores y de Studio.
`Indexador propio (script + base de datos)`	Necesidades muy específicas o datos fuera de la cadena.	Operar un servidor y una base de datos.	Todo: ustedes son el punto único.

REGLA DEL CURSO

Usen la pieza más simple que resuelva. Para el Avance 2 basta queryFilter en casi todos los proyectos. Si eligen un subgrafo, el README debe decir por qué queryFilter no alcanzaba: es una decisión de diseño que se defiende.

ENS: VITALIK.ETH EN VEZ DE 0XD8DA...6045

Una dirección de 42 caracteres no se puede verificar a ojo. ENS es un registro en cadena que asocia nombres a direcciones, como el DNS asocia dominios a IP. Resolverlo es una LECTURA en dos pasos.

RESUELTO DE VERDAD EL 17-SEP-2026, RED PRINCIPAL

```
const p = new ethers.JsonRpcProvider(URL_MAINNET);    // la URL, desde el keystore
await p.resolveName("vitalik.eth");
//  → 0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045        (nombre → dirección)
await p.lookupAddress("0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045");
//  → "vitalik.eth"                                    (dirección → nombre, «reverse»)
```

CÓMO FUNCIONA

El registro ENS dice qué contrato «resolver» atiende cada nombre; el resolver devuelve la dirección, o un contenthash para un sitio en IPFS.

ERROR TÍPICO

Creer que el nombre es para siempre: los .eth se pagan por año (5 USD/año desde 5 letras) y, si vencen, tras 90 días de gracia otro los puede registrar.

TRES FORMAS DE FIRMAR, TRES RIESGOS

En la Sesión 3 firmamos transacciones. Pero una billetera también firma MENSAJES: no van a la cadena, no cuestan gas, y aun así prueban quién los autorizó. La diferencia está en qué ve el usuario antes de firmar.

TIPO	QUÉ SE FIRMA	QUÉ VE EL USUARIO	RIESGO
Hash crudo (eth_sign)	32 bytes cualesquiera.	Un hexadecimal ilegible.	Puede ser el hash de una transacción: por eso las billeteras lo bloquean.
Texto (ERC-191, personal_sign)	Un texto con el prefijo «\x19Ethereum Signed Message».	El texto tal cual.	El prefijo impide que sea una transacción. Es el que usa SIWE.
Datos tipados (EIP-712)	Campos con tipo + un DOMINIO (app, versión, red, contrato).	Cada campo con su nombre y valor.	Si el usuario no lee los campos, puede firmar un permiso de gastar sus tokens.

La firma sigue siendo la de la Sesión 3: ECDSA sobre secp256k1, y de la firma se recupera la dirección. Lo que cambia es QUÉ bytes se firman.

DOMINIO + TIPOS + MENSAJE = UN RESUMEN

CÓMO SE FIRMA Y SE VERIFICA · ETHERS 6

```
const dominio = { name: "AsistenciaUSB", version: "1",
                  chainId: 11155111, verifyingContract: "0x…" };   // Sepolia y SU contrato
const tipos = { Asistencia: [
  { name: "estudiante", type: "address" }, { name: "sesion", type: "uint256" },
  { name: "nonce",      type: "uint256" }, { name: "vence",  type: "uint256" } ] };
const firma = await signer.signTypedData(dominio, tipos, mensaje);   // gratis
ethers.verifyTypedData(dominio, tipos, mensaje, firma);              // → dirección
```

```
resumen = keccak256( 0x19 0x01  ‖  hash(dominio)  ‖  keccak256( typeHash ‖ estudiante ‖ sesion ‖ nonce ‖ vence ) )
```

EN SOLIDITY (OPENZEPPELIN 5)

Heredan EIP712("AsistenciaUSB", "1"). _hashTypedDataV4(hashStruct) arma el resumen; ECDSA.recover(resumen, firma) devuelve quién firmó.

ERROR TÍPICO

Escribir el tipo con un espacio de más, otro orden o abi.encodePacked. La firma «no verifica» y nadie sabe por qué: el typeHash debe ser idéntico, carácter por carácter.

UNA FIRMA VÁLIDA NO ES VÁLIDA PARA SIEMPRE

Una firma es un objeto que cualquiera puede copiar. Si el mensaje no dice DÓNDE, CUÁNTAS VECES y HASTA CUÁNDO vale, quien la copie la puede reenviar. Cada campo cierra un ataque concreto, y la práctica prueba los cinco:

ATAQUE	QUÉ LO IMPIDE	PRUEBA DE LA PRÁCTICA
Reenviar la firma en otra red	chainId en el dominio.	★ dominio de otra cadena
Reenviarla a otro contrato	verifyingContract en el dominio.	★ dominio de otro contrato
Enviarla dos veces	nonce que se consume al usarla.	★ repetición (InvalidAccountNonce)
Usarla meses después	vence: fecha límite en el mensaje.	★ firma vencida
Firmar por otra persona	recover(resumen, firma) ≠ estudiante.	★ firma de otro (FirmaInvalida)

Error típico: comprobar la firma y olvidar el nonce. El contrato «funciona» en todas las pruebas felices y es vulnerable a la primera repetición.

FIRMAR UN PERMISO ES ENTREGAR LA LLAVE

EIP-712 hace legible la firma, pero no la hace inofensiva. El mismo mecanismo que registra una asistencia sirve, en ERC-2612 (permit), para autorizar a un tercero a gastar tus tokens. Sin gas, sin transacción visible.

EL ATAQUE DE PHISHING

- Un sitio falso pide «firmar para verificar la billetera».
- Lo que se firma es un Permit: spender = el atacante, value = todo.
- El atacante envía la firma y vacía los tokens. La víctima nunca pagó gas.

LA COSTUMBRE PROFESIONAL

- Leer el dominio: ¿es la app que creo, en la red que creo?
- Leer el tipo: Permit, Order, Delegation… ¿qué autoriza?
- Si no se entiende, no se firma. Regla del curso desde la S1.

REGLA DEL CURSO

No se firma lo que no se entiende, aunque «sea gratis». Una firma sin gas puede mover más dinero que una transacción.

SIGN-IN WITH ETHEREUM (ERC-4361)

EL MENSAJE ESTÁNDAR (TEXTO, ERC-191)

```
miapp.edu.co wants you to sign in with your Ethereum account:
0x70997970C51812dc3A010C7d01b50e0d17dc79C8

Ingresar al tablero del curso.

URI: https://miapp.edu.co
Version: 1
Chain ID: 11155111
Nonce: 8f3kQ2mZ
Issued At: 2026-09-17T19:30:00Z
Expiration Time: 2026-09-17T19:40:00Z
```

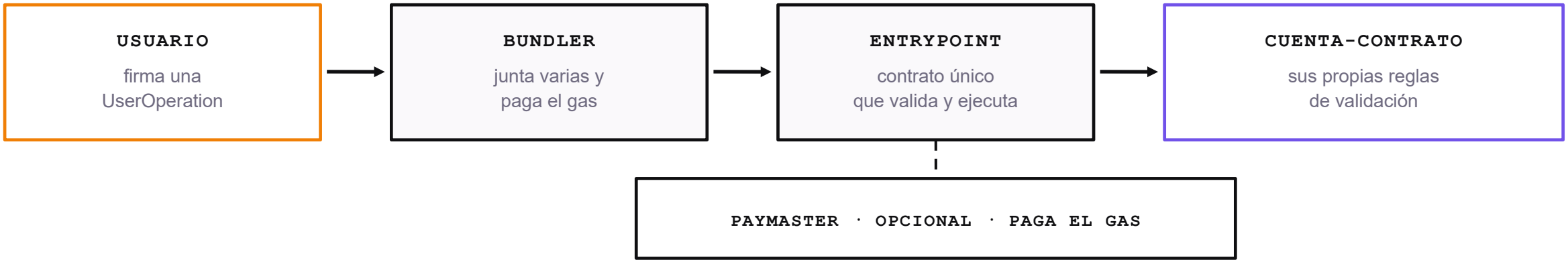
01	NONCE	El servidor genera un nonce de un solo uso.
02	FIRMA	La billetera muestra el texto; el usuario firma.
03	VERIFICA	El servidor recupera la dirección y revisa dominio, nonce y fecha.
04	SESIÓN	Emite una cookie o un token como siempre.

⚠ **ERROR TÍPICO: «SIWE ES EIP-712»**

No: ERC-4361 firma TEXTO con ERC-191, a propósito, porque todas las billeteras lo soportan. EIP-712 es otra herramienta, la que usa la práctica del viernes 6 para autorizar una acción en un contrato.

ABSTRACCIÓN DE CUENTAS: ERC-4337

Una cuenta normal (EOA) es «una clave, una cuenta»: sin la frase semilla no hay recuperación, y sin ETH no hay primera transacción. ERC-4337 convierte la cuenta en un contrato programable, sin cambiar el protocolo.



CAPACIDAD	QUÉ RESUELVE PARA EL USUARIO NUEVO
Recuperación social o por guardianes	Perder un dispositivo ya no es perderlo todo.
Gas patrocinado (paymaster)	Empezar sin tener ETH: la app paga.
Operaciones por lotes	approve + usar en una sola confirmación (el doble paso del bloque A).

EIP-7702: LA CUENTA NORMAL APRENDE TRUCOS

Con la actualización Pectra (red principal, 7 de mayo de 2025) una EOA puede firmar una AUTORIZACIÓN que delega su ejecución al código de un contrato. Conserva la misma dirección y los mismos fondos, y gana lotes y gas patrocinado.

	ERC-4337	EIP-7702
Qué es	Estándar de contratos, sin cambiar el protocolo.	Cambio del protocolo: un tipo de transacción nuevo.
La dirección	Una cuenta-contrato nueva, con dirección nueva.	La misma EOA de siempre.
La clave original	Puede dejar de ser necesaria (otras reglas).	Sigue mandando: quien tiene la clave, manda.
Se complementan	Aporta la infraestructura: bundlers, paymasters.	Deja que una EOA existente use esa infraestructura.

EL RIESGO NUEVO

Firmar una autorización 7702 hacia un contrato malicioso es entregarle la cuenta entera. Es la versión extrema de la lámina C.11: la billetera debe mostrar a qué código se delega, y el usuario debe entenderlo.

IPFS + ENS FRENTE A HOSTING TRADICIONAL

HOSTING TRADICIONAL

Vercel, Netlify o un servidor propio. Rápido de montar y gratis para empezar. Pero si el proveedor cae, bloquea la cuenta o cambia el código, la dApp deja de estar accesible o sirve otra cosa.

IPFS + ENS

Se compila el frontend, se sube a IPFS (contenido direccionado por hash) y se pone su CID en el contenthash del nombre ENS. Un gateway como eth.limo lo sirve: vitalik.eth.limo funciona así hoy.

CAPA	¿QUIÉN LA CONTROLA EN UNA DAPP TÍPICA?
Contratos	Nadie en particular: la cadena.
Frontend	El proveedor de hosting, o nadie si está en IPFS con su CID fijado (pinning).
Gateway IPFS y RPC	Empresas concretas: siguen siendo puntos centrales.
Indexador	Studio / red de The Graph, o su propio servidor.

SEIS IDEAS DE INFRAESTRUCTURA

01 El estado guarda el presente; la historia está en los eventos. Se lee con queryFilter o se indexa.

02 The Graph hoy se usa por Subgraph Studio: el hosted service se apagó en junio de 2024.

03 ENS es una lectura en dos pasos; los nombres vencen y los parecidos engañan.

04 EIP-712 firma datos con dominio; el nonce y el vencimiento evitan que la firma se reuse.

05 SIWE firma texto (ERC-191). 4337 y 7702 hacen la cuenta programable; 7702 ya está en la red.

06 Un repo está listo cuando otro lo corre siguiendo el README, sin secretos dentro.

LO QUE SE LLEVAN DE HOY

Token: todos saben hablarle. NFT: importa dónde vive su archivo. dApp: la capa que une a ambos.

La próxima clase, todo esto en las manos: un lab en Hardhat local donde una ficha ERC-20 se canjea por un diploma NFT intransferible — sin ETH, sin red pública.

VIERNES 6 DE NOVIEMBRE · PRÁCTICA INTEGRADA EN HARDHAT LOCAL